

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	K. Dedeepya	Reg. No.:	192324252
Section / Batch:	AIDS SLOT-B	Date:	16-08-2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & SyntaxDriven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyse how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

Ans:

Context-Free Grammar (CFG) trees and dependency parsing represent sentence structure in different ways.

CFG represents a sentence using a hierarchical phrase structure.

For example: “The student reads the book.”

The sentence can be divided into:

S → NP + VP

NP → The student

VP → reads + NP

NP → the book

This representation shows how words form larger grammatical phrases.

Dependency parsing represents the same sentence through direct relationships between words. The main verb “reads” acts as the head. “student” is connected to “reads” as the subject, and “book” is connected as the object.

CFG is useful for identifying grammatical constituents such as noun phrases and verb phrases. However, it does not directly show the relationship between every pair of words.

Dependency parsing directly represents relationships such as:

- Subject
- Object
- Modifier
- Head
- Dependent

Therefore, dependency parsing is more effective when the main requirement is to identify relationships between words.

For grammar checking, CFG provides useful hierarchical information. For semantic analysis, information extraction, question answering, and relation extraction, dependency structures are often more useful because they directly represent word relationships.

Conclusion: CFG is better for hierarchical grammatical structure, while dependency parsing is better for direct word-to-word relationships. For applications that need to understand relationships between words, dependency parsing is the more effective approach.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyse the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

Ans:

Top-down parsing starts from the grammar's start symbol and attempts to generate a structure that matches the input sentence.

For example, if the grammar starts with: $S \rightarrow NP VP$

the parser first tries to identify the noun phrase and then the verb phrase.

Top-down parsing is relatively simple and works well for predictable grammatical structures. However, when the input is ambiguous, it may try several alternatives and perform repeated backtracking.

Earley parsing uses a chart-based approach and processes the input incrementally. It maintains partial parsing states, allowing it to handle incomplete and ambiguous input more effectively.

For a real-time system, a user may provide an incomplete command such as: "Book a flight to..."

The system can maintain a partial interpretation while waiting for the remaining input.

Top-down parsing may struggle because the complete grammatical structure has not yet been received.

Earley parsing is more flexible because it can maintain partial structures.

Technical Comparison

Top-down parsing:

- Simple implementation.
- Works well with simple grammars.
- May require backtracking.
- Less suitable for highly ambiguous input.
- Can become inefficient with many alternatives.

Earley parsing:

- Handles general CFGs.
- Supports ambiguous structures.
- Maintains partial parses.
- Reduces unnecessary repeated parsing.
- Suitable for incomplete and dynamic input.

Earley parsing has a general worst-case complexity of $O(n^3)$, while it can perform better for many practical grammars. Its chart structure avoids repeatedly recomputing the same partial parses.

Conclusion: Top-down parsing is suitable for simple and predictable inputs. For real-time systems involving incomplete, ambiguous, or dynamically changing input, Earley parsing is more suitable because it supports partial parsing and handles ambiguity more systematically.

3. An NLP model must handle ambiguity in sentences such as "She saw the man with a telescope." The designers are considering CFG, PCFG, and neural parsing techniques.

Analyse how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

Ans:

Consider the sentence: "She saw the man with a telescope."

The sentence has two possible interpretations.

Interpretation 1: She used a telescope to see the man.

Interpretation 2: She saw a man who had a telescope.

The ambiguity occurs because “with a telescope” can modify the verb phrase or the noun phrase.

CFG:

CFG represents the possible syntactic structures using grammar rules.

It can generate both interpretations when the grammar permits both structures.

However, basic CFG does not contain probabilities. Therefore, it cannot determine which interpretation is more likely.

Advantage: Clear and interpretable grammatical structure.

Limitation: Poor ambiguity resolution when multiple valid parses exist.

PCFG:

Probabilistic Context-Free Grammar extends CFG by assigning probabilities to grammar rules.

If training data indicates that “with a telescope” is more commonly attached to the verb phrase in a particular context, that parse can receive a higher probability.

Therefore, PCFG can select the most probable parse instead of treating all valid parses equally.

Advantage: Better ambiguity resolution than basic CFG.

Limitation: Depends strongly on the quality and representativeness of the training data.

Neural Parsing:

Neural parsers use machine-learning models to learn contextual relationships from large language datasets.

They can use information from surrounding words and broader context.

For example: “She used her telescope and saw the man.”

clearly indicates that the telescope is associated with the action of seeing.

Neural parsing can capture such contextual information more effectively than traditional rule-based methods.

Advantages:

- Strong contextual understanding.
- Handles complex language patterns.
- Usually provides high parsing accuracy with sufficient training data.
- Suitable for modern NLP applications.

Limitations:

- Requires large training datasets.
- Requires greater computational resources.
- Can be less interpretable than rule-based approaches.

Final Comparison

CFG is mainly concerned with grammatical structure.

PCFG adds probabilities to resolve competing grammatical structures.

Neural parsing uses learned contextual representations to make more sophisticated parsing decisions.

For simple grammar systems, CFG is appropriate. For probabilistic ambiguity resolution, PCFG is useful. For modern real-world applications involving complex and context-dependent language, neural parsing is generally the most effective, provided sufficient training data and computational resources are available.

Conclusion: Neural parsing provides the strongest overall capability for real-world ambiguity resolution, while CFG and PCFG remain valuable when interpretability, controlled grammars, or probabilistic grammatical analysis are required.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyse how these approaches differ and justify which provides better support for enforcing grammatical correctness.

Ans:

Feature structures and subcategorization frames address different aspects of linguistic representation.

Feature Structures

A feature structure represents grammatical properties associated with linguistic units.

Important features include:

- Number
- Person
- Tense
- Gender
- Case
- Agreement

For example: “The student reads the book.”

The subject student is singular.

Therefore, the verb should also have the singular feature:

student → Number: Singular

reads → Number: Singular

The agreement constraint is satisfied.

If the sentence were: “The student read the books.”

the system can analyse the tense and number features to determine whether the grammatical relationship is valid according to the grammar being used.

Feature structures are therefore highly useful for enforcing grammatical agreement.

Subcategorization Frames

Subcategorization frames describe the arguments that a particular verb requires.

For example:

eat → Subject + Verb + Object

Sentence: “The patient takes medicine.”

Here:

Patient → Subject

takes → Verb

medicine → Object

Another example:

give → Subject + Verb + Object + Recipient

Sentence: “The doctor gave the patient medicine.”

The verb gave requires an appropriate argument structure involving the giver, object, and recipient.

Subcategorization frames therefore help determine whether a verb has received the appropriate arguments.

Comparison

Feature structures are stronger for grammatical constraints and agreement.

Subcategorization frames are stronger for verb argument structures and semantic relationships.

For example, feature structures can check whether subject and verb features agree, while a subcategorization frame can determine whether a verb such as “give” requires an object and recipient. Therefore, neither method completely replaces the other.

Conclusion: If the primary requirement is enforcing subject–verb agreement, feature structures provide better support. If the requirement is representing verb argument structures, subcategorization frames are better. A high-quality grammar system should combine both for accurate grammatical and semantic analysis.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyse the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Ans:

Transition-based parsing and graph-based parsing are two important approaches to dependency parsing.

Transition-Based Parsing

Transition-based parsing constructs a dependency tree through a sequence of decisions.

Typical operations include:

- Shift
- Reduce
- Left-Arc
- Right-Arc

The parser processes the sentence incrementally and decides which dependency should be created at each step.

Its main advantage is speed because decisions are made locally and incrementally.

However, an incorrect early decision can affect later decisions. This can cause error propagation.

Graph-Based Parsing

Graph-based parsing considers possible dependency relationships as a graph.

It searches for the dependency tree with the highest overall score.

Instead of making only local decisions, it evaluates relationships more globally.

Its main advantage is that it can produce more globally consistent dependency structures.

However, graph-based methods generally require more computation than transition-based methods, particularly when processing large datasets.

Performance Comparison

Transition-based parsing:

- Fast processing.
- Low computational cost.
- Suitable for real-time applications.
- Scales well to large datasets.
- May suffer from error propagation.

Graph-based parsing:

- Considers global dependency structure.
- More robust against some local decision errors.
- Can provide strong overall parsing accuracy.
- More computationally expensive.

- May be slower for massive datasets.

Industry Application

For a large-scale NLP system processing millions of sentences, processing speed and scalability are important.

Transition-based parsing is generally more suitable when the system requires high throughput and low latency.

Graph-based parsing is more appropriate when global parsing accuracy is prioritized over processing speed and sufficient computational resources are available.

Conclusion: For large-scale applications where fast and efficient dependency parsing is essential, transition-based parsing is generally the better choice. Graph-based parsing is preferred when global structural accuracy is more important than speed.

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant

Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand